Memoria: Procesamiento de datos meteorológicos con MapReduce

Apache Hadoop y Apache Spark

1. Apache Hadoop

1.1. Preparación

Todo lo necesario para esta parte (datos de entrada y scripts de mapper/reducer) está contenido en la carpeta `ics_map_reduce/`. Lo primero es copiar esa carpeta **entera** dentro del contenedor `namenode` del clúster Hadoop, en lugar de copiar los ficheros uno a uno:

```
docker compose cp ics_map_reduce namenode:/tmp/ics_map_reduce
```

Este comando copia de golpe todo el contenido del proyecto (datos meteorológicos y scripts) dentro del contenedor, en `/tmp/ics_map_reduce`. A partir de aquí, todos los comandos siguientes se ejecutan dentro del contenedor y hacen referencia a los ficheros usando rutas **relativas** a esa copia, sin necesidad de volver a copiar nada.

Nota: los mappers y reducers están escritos en Python 3. La imagen Docker del clúster se ha extendido (ver `Dockerfile` del proyecto) para instalar Python 3 sobre la imagen base `apache/hadoop:3`, que originalmente solo traía Python 2.

1.2. Pasos

1. **Esperar a que HDFS salga del modo seguro** (justo después de levantar el clúster con `just up`):

   ```
   docker compose exec -T namenode hdfs dfsadmin -safemode wait
   ```

   Al arrancar, el `namenode` entra en *safe mode* mientras recibe los informes de bloques de los cuatro `datanode`; en ese estado HDFS es de solo lectura y no admite escrituras. Este comando se queda esperando hasta que el clúster sale del modo seguro, momento en el que ya se pueden subir datos.

2. **Crear el directorio de usuario en HDFS** (una sola vez):

   ```
   docker compose exec -T namenode hdfs dfs -mkdir -p /ics2026ms
   ```

   Es el directorio dentro del sistema de ficheros distribuido (HDFS) donde se guardarán los datos de entrada y los resultados de este usuario.

3. **Subir los datos de entrada a HDFS**, usando la ruta relativa a la copia hecha en el paso anterior:

   ```
   docker compose exec -T namenode hdfs dfs -mkdir -p /ics2026ms/
       datos_216_estaciones
   ```

```
docker compose exec -T namenode bash -c "hdfs dfs -put -f /tmp/ics_map_reduce/
    datos/datos_216_estaciones/* /ics2026ms/datos_216_estaciones/"
```

Sube los ficheros JSON de las estaciones meteorológicas desde la copia local dentro del contenedor hacia HDFS, que es el sistema de ficheros distribuido desde el que los distintos nodos del clúster leerán los datos en paralelo.

4. **Lanzar el job de streaming de Hadoop**, apuntando el mapper y el reducer directamente a la carpeta copiada:

```
docker compose exec -T namenode bash -c "hadoop jar /opt/hadoop/share/hadoop/
    tools/lib/hadoop-streaming-*.jar \
    -files /tmp/ics_map_reduce/hadoop/map_hadoop.py,/tmp/ics_map_reduce/hadoop/
        reduce_hadoop.py \
    -mapper 'python3 map_hadoop.py' -reducer 'python3 reduce_hadoop.py' \
    -input /ics2026ms/datos_216_estaciones -output /ics2026ms/salida"
```

Significado de cada opción:

- `hadoop jar .../hadoop-streaming-*.jar`: invoca el framework *Hadoop Streaming*, que permite escribir el mapper y el reducer en cualquier lenguaje (aquí Python) en vez de Java, comunicándose por stdin/stdout.
- `-files`: distribuye los dos scripts a todos los nodos (*nodemanagers*) que vayan a ejecutar tareas, ya que el procesamiento real no ocurre en el `namenode`.
- `-mapper`: comando que se ejecuta en la fase *map*; lee líneas de la entrada por stdin y extrae de cada registro JSON el nombre de estación, `TMAX`, `TMIN` y fecha.
- `-reducer`: comando que se ejecuta en la fase *reduce*; recibe las salidas de los mappers ya agrupadas y ordenadas por clave (estación) y calcula la temperatura máxima y mínima de cada una.
- `-input` / `-output`: rutas en HDFS de los datos de entrada y del directorio donde se escribirá el resultado (este último no debe existir previamente).

5. **Consultar el resultado**:

```
docker compose exec -T namenode hdfs dfs -ls /ics2026ms/salida
docker compose exec -T namenode hdfs dfs -cat /ics2026ms/salida/part-00000
```

Hadoop Streaming usa la API antigua de MapReduce, por lo que el fichero de salida se llama `part-00000` (sin el sufijo `-r-` que usa la API nueva).

1.3. Escalado (opcional)

El número de *reducers* no depende del número de nodos del clúster; por defecto es 1 y se fuerza explícitamente con una opción genérica:

```
docker compose exec -T namenode bash -c "hadoop jar /opt/hadoop/share/hadoop/tools/lib
    /hadoop-streaming-*.jar \
    -D mapreduce.job.reduces=4 \
    -files /tmp/ics_map_reduce/hadoop/map_hadoop.py,/tmp/ics_map_reduce/hadoop/
        reduce_hadoop.py \
    -mapper 'python3 map_hadoop.py' -reducer 'python3 reduce_hadoop.py' \
    -input /ics2026ms/datos_216_estaciones -output /ics2026ms/salida"
```

El número de *mappers* tampoco se fija a mano: depende de cuántos *splits* genera el formato de entrada. Con muchos ficheros pequeños (p. ej. 216 ficheros de datos JSON) Hadoop crea por defecto un split por fichero, generando muchas tareas diminutas. Para combinarlos en menos mappers se usa `CombineTextInputFormat` junto con un tamaño máximo de split:

```
docker compose exec -T namenode bash -c "hadoop jar /opt/hadoop/share/hadoop/tools/lib
    /hadoop-streaming-*.jar \
    -D mapreduce.job.reduces=4 \
    -D mapreduce.input.fileinputformat.split.maxsize=3200000 \
    -files /tmp/ics_map_reduce/hadoop/map_hadoop.py,/tmp/ics_map_reduce/hadoop/
        reduce_hadoop.py \
    -inputformat org.apache.hadoop.mapred.lib.CombineTextInputFormat \
    -mapper 'python3 map_hadoop.py' -reducer 'python3 reduce_hadoop.py' \
    -input /ics2026ms/datos_216_estaciones -output /ics2026ms/salida"
```

Importante: las opciones genéricas (`-D`, `-files`) deben ir siempre antes que las opciones propias de streaming (`-inputformat`, `-mapper`, `-reducer`, `-input`, `-output`); si se invierte el orden, el parser deja de reconocer las opciones genéricas y el job falla.

2. Apache Spark

2.1. Requisitos

Se necesita tener instalado `pyspark` en el entorno desde el que se va a lanzar el job:

```
pip install pyspark
```

El comando `spark-submit` (incluido con `pyspark`) permite ejecutar el script directamente, sin necesidad de un clúster Hadoop en marcha.

2.2. Pasos

1. **Script de partida**: `ics_map_reduce/spark/map_reduce_temp.py`. Este script:
   - En la fase *map*, usa `sc.wholeTextFiles(sys.argv[1])` para leer cada fichero JSON de la carpeta de datos brutos como un par (ruta, contenido completo), y un `flatMap` que recorre cada registro del array JSON, descarta los que no tengan `TMAX`/`TMIN` válidos, y emite pares `(nombre_estación, (tmax, fecha, tmin, fecha))`.
   - En la fase *reduce*, usa `reduceByKey` con `numPartitions=2` (dos particiones, equivalentes a dos reducers de Hadoop) para quedarse, por estación, con la temperatura máxima y la mínima.
   - Escribe el resultado de cada partición en un fichero (`part-00000.txt`, `part-00001.txt`), y un resumen final con la estación más calurosa y la más fría, dentro de `ics_map_reduce/spark/resultados_temperaturas/`.
2. **Ejecutar el script**, pasándole como argumento la carpeta con los datos brutos (los mismos ficheros JSON de estaciones meteorológicas usados en la parte de Hadoop):

   ```
   spark-submit ics_map_reduce/spark/map_reduce_temp.py ics_map_reduce/datos/
       datos_4_estaciones
   ```

   El argumento `sys.argv[1]` es la ruta (local) a la carpeta de datos; Spark la recorre entera, leyendo cada fichero JSON como un registro completo.
3. **Consultar el resultado**, generado localmente junto al script:

```
cat ics_map_reduce/spark/resultados_temperaturas/part-00000.txt
cat ics_map_reduce/spark/resultados_temperaturas/part-00001.txt
cat ics_map_reduce/spark/resultados_temperaturas/resumen.txt
```

Los dos primeros ficheros contienen el resultado de cada una de las dos particiones/reducers (estación, temperatura máxima y fecha, temperatura mínima y fecha). El fichero `resumen.txt` indica la estación con la temperatura más alta y la más baja de todo el conjunto de datos.